



INTELLIGENT SENSORY SIGNAL PROCESSING USING MACHINE LEARNING

DATA-DRIVEN & REPRESENTATION LEARNING

Dr TIAN Jing

tianjing@nus.edu.sg

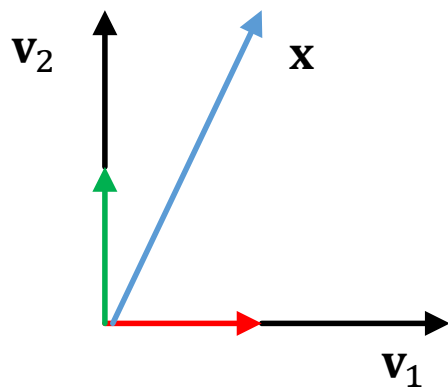


DATA-DRIVEN REPRESENTATION FOR SENSORY DATA



Warm-up: Basis vectors

- A given vector value is represented in a *coordinate system*.
- A coordinate system is defined by a set of linearly independent vectors forming the system's *basis*.
- Any vector value is represented as a linear sum of the basis vectors.
- **Key idea:** The basis vector determines how the signal is represented. We can change basis vectors so that we can represent the signal in a different representation.

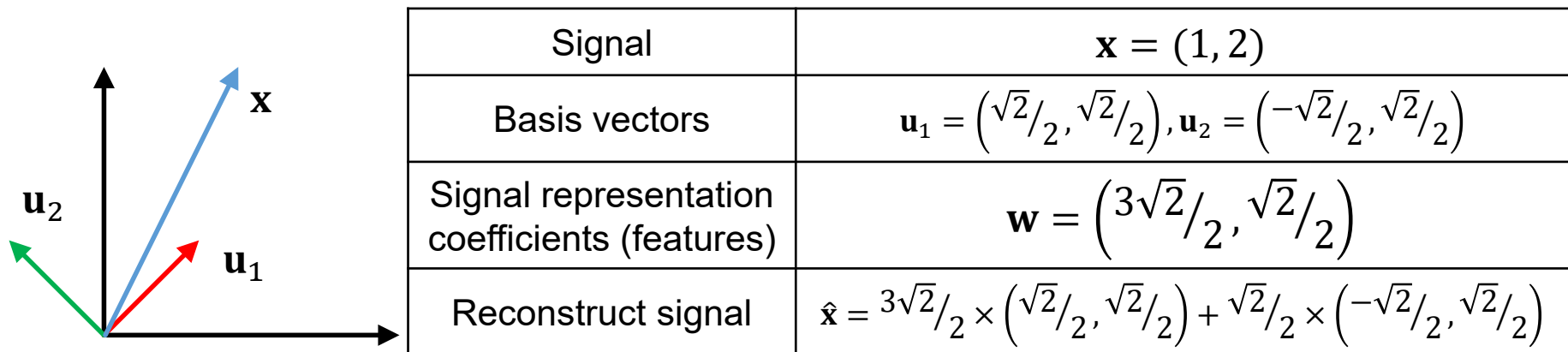


Signal	$\mathbf{x} = (1, 2)$
Basis vectors	$\mathbf{v}_1 = (1, 0), \mathbf{v}_2 = (0, 1)$
Signal representation coefficients (features)	$\mathbf{w} = (1, 2)$



Warm-up: Change of basis vectors

- Question:** Given a signal $\mathbf{x}$, represented in orthonormal basis vectors $\mathbf{v}_1, \mathbf{v}_2$, what is the representation of $\hat{\mathbf{x}}$ in a different orthonormal basis vectors $\mathbf{u}_1, \mathbf{u}_2$?



- Decompose signal $\mathbf{x}$

$$w_i = \langle \mathbf{x}, \mathbf{u}_i \rangle = \mathbf{x}^T \mathbf{u}_i = \sum_j x(j) u_i(j)$$

where $\langle \cdot \rangle$ is the dot product of two vectors

- Reconstruct signal $\hat{\mathbf{x}}$

$$\hat{\mathbf{x}} = \sum_i w_i \times \mathbf{u}_i$$

Appendix: higher dimensions (checkerboard basis)

Signal at standard basis: $\mathbf{x} = \begin{bmatrix} 2 & 1 \\ 6 & 1 \end{bmatrix}$ 

$$\begin{bmatrix} 2 & 1 \\ 6 & 1 \end{bmatrix} = \textcolor{red}{2} \times \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix} + \textcolor{red}{1} \times \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix} + \textcolor{red}{6} \times \begin{bmatrix} 0 & 0 \\ 1 & 0 \end{bmatrix} + \textcolor{red}{1} \times \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix}$$

New basis

$$\mathbf{u}_1 = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} / 2 \quad \mathbf{u}_2 = \begin{bmatrix} 1 & 1 \\ -1 & -1 \end{bmatrix} / 2$$

$$\mathbf{u}_3 = \begin{bmatrix} -1 & 1 \\ 1 & -1 \end{bmatrix} / 2 \quad \mathbf{u}_4 = \begin{bmatrix} -1 & 1 \\ -1 & 1 \end{bmatrix} / 2$$

Recall the formula

$$w_i = \langle \mathbf{x}, \mathbf{u}_i \rangle = \mathbf{x}^T \mathbf{u}_i = \sum_j x(j) u_i(j)$$

where $\langle \cdot \rangle$ is the dot product of two vectors

$$\hat{\mathbf{x}} = \sum_i w_i \times \mathbf{u}_i$$

Signal representation coefficients

at new basis: $\mathbf{w} = \begin{bmatrix} \textcolor{red}{5} & -2 \\ \textcolor{orange}{2} & \textcolor{orange}{-3} \end{bmatrix}$

$$\begin{aligned} \langle \mathbf{x}, \mathbf{u}_1 \rangle &= \sum_j x(j) u_1(j) \\ &= \left(2 * \frac{1}{2} + 1 * \frac{1}{2} + 6 * \frac{1}{2} + 1 * \frac{1}{2} \right) = \textcolor{red}{5} \end{aligned}$$

$$\begin{aligned} \langle \mathbf{x}, \mathbf{u}_4 \rangle &= \sum_j x(j) u_4(j) \\ &= \left(2 * \frac{-1}{2} + 1 * \frac{1}{2} + 6 * \frac{-1}{2} + 1 * \frac{1}{2} \right) = \textcolor{orange}{-3} \end{aligned}$$

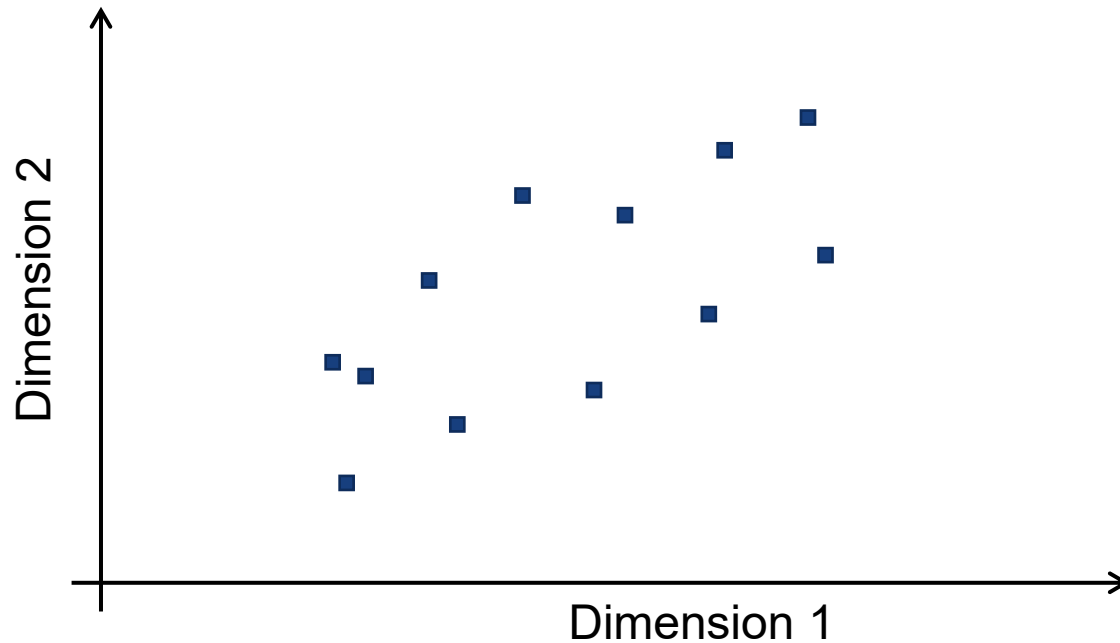


Idea: Energy compaction property

- Assumption: There are a set of N typical basis vectors that captures most of all (say, M) input data $\mathbf{x}_i$, where $i = 1, \dots, M$.
- Given the signal representation as $\hat{\mathbf{x}} = \sum_{k=1}^N w_k \mathbf{u}_k$

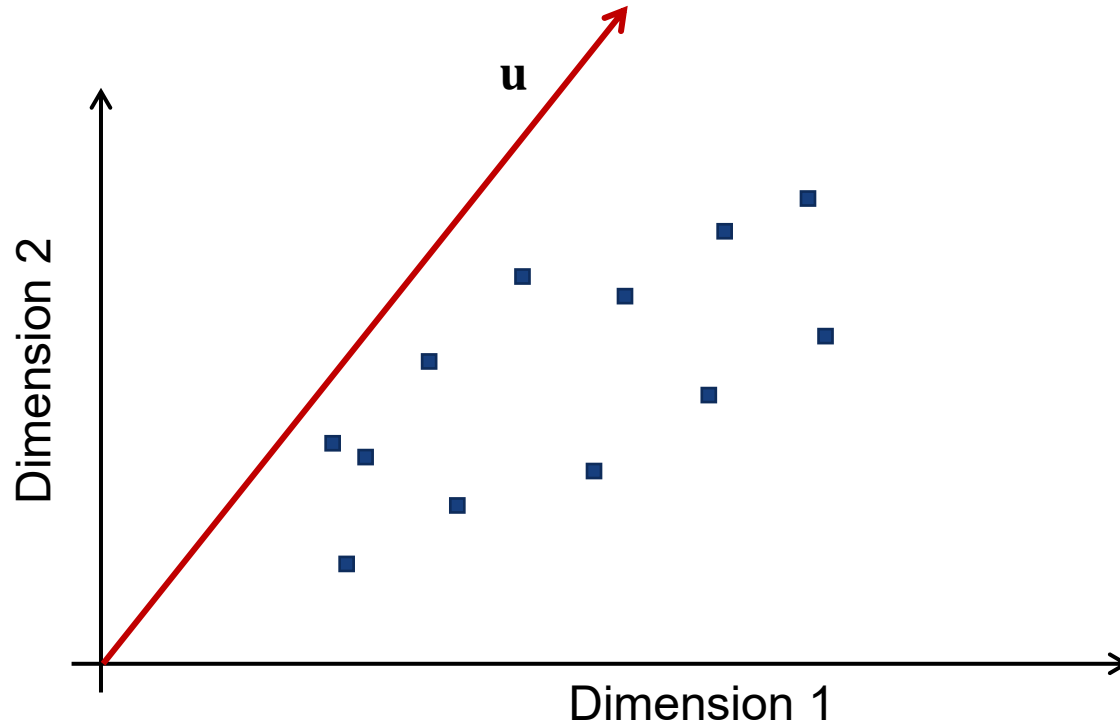
Note: N might not be as same as the dimension of $\mathbf{X}$
- That is $\hat{\mathbf{x}} = w_1 \mathbf{u}_1 + w_2 \mathbf{u}_2 + w_3 \mathbf{u}_3 + \dots + w_N \mathbf{u}_N$ based on N basis components. Its error is defined as $\text{Error}_N = \|\mathbf{x} - \hat{\mathbf{x}}\|^2$
- Approximate every data $\mathbf{x}_i$ as $\hat{\mathbf{x}}_i = w_{i,1} \mathbf{u}_1 + w_{i,2} \mathbf{u}_2 + \dots + w_{i,N} \mathbf{u}_N$
 - $\mathbf{u}_2$ is used to “correct” errors resulting from using only $\mathbf{u}_1$.
 - $\|\mathbf{x}_i - (w_{i,1} \mathbf{u}_1 + w_{i,2} \mathbf{u}_2)\|^2 < \|\mathbf{x}_i - w_{i,1} \mathbf{u}_1\|^2$
 - $\mathbf{u}_3$ corrects errors remaining after correction with $\mathbf{u}_2$
 - $\|\mathbf{x}_i - (w_{i,1} \mathbf{u}_1 + w_{i,2} \mathbf{u}_2 + w_{i,3} \mathbf{u}_3)\|^2 < \|\mathbf{x}_i - (w_{i,1} \mathbf{u}_1 + w_{i,2} \mathbf{u}_2)\|^2$
 - And so on
- Estimate $\mathbf{u}_1, \mathbf{u}_2, \dots, \mathbf{u}_N$ to minimize the squared error between the original signal and the reconstructed signal.

Find the first basis vector



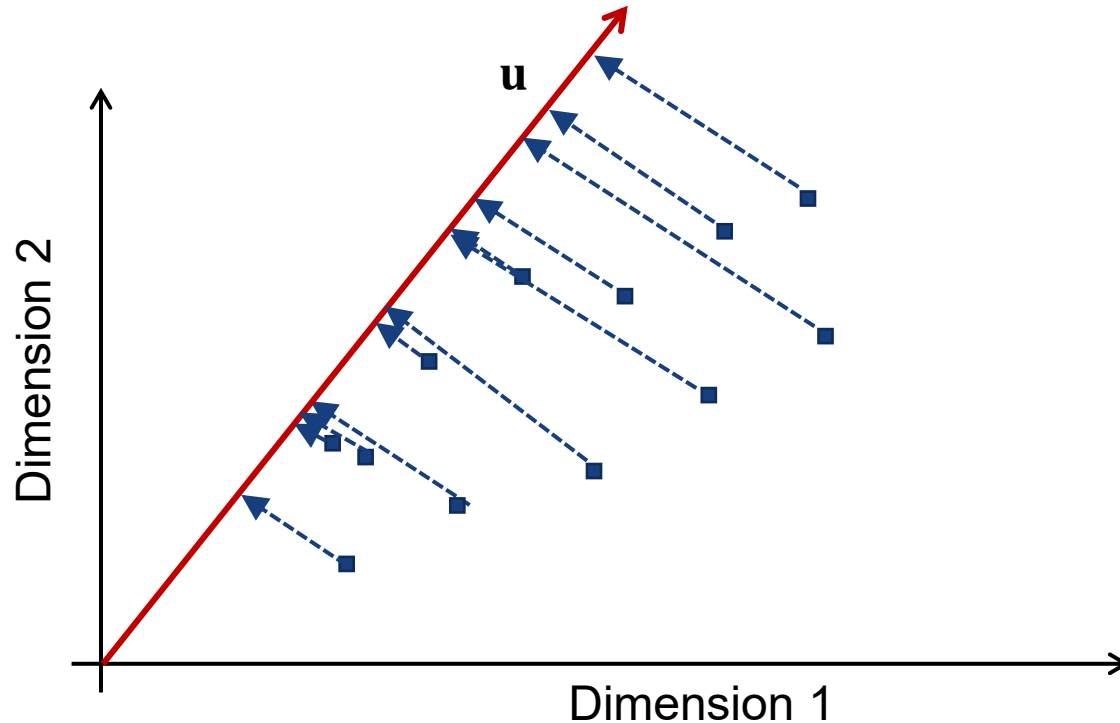
- Each “point” represents a signal data.

Find the first basis vector



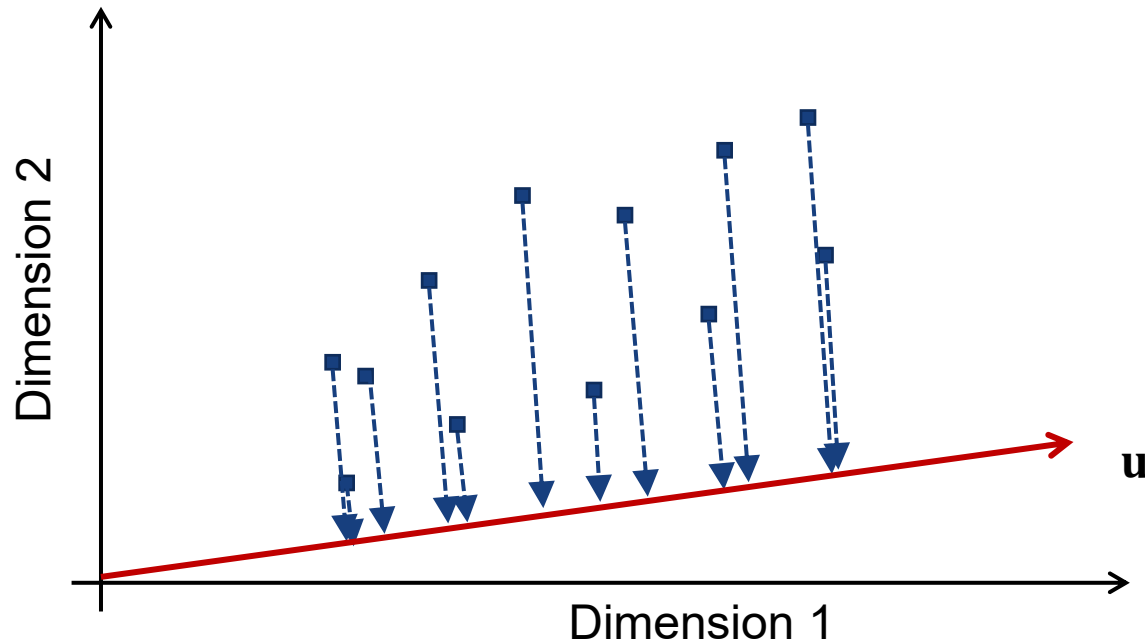
- Each “point” represents a signal data.
- Any “basis vector” u is a vector in this space.

Find the first basis vector



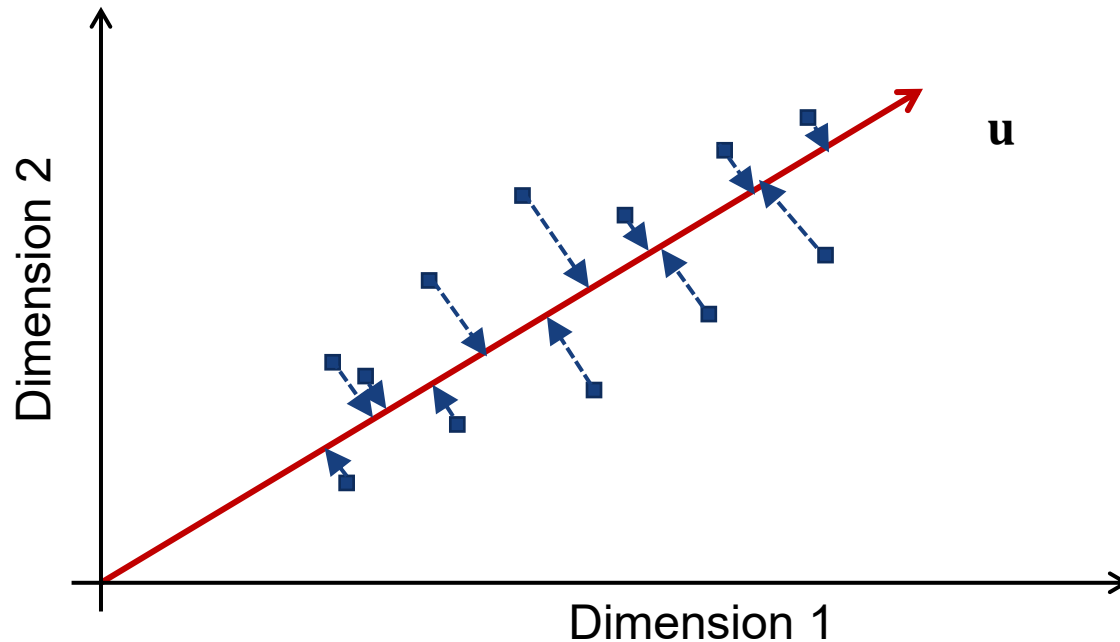
- Every signal data will suffer an error when approximated by its projection on $\mathbf{u}$.
- The total length of all error lines is the *total projection error*.
- Objective: Find the best $\mathbf{u}$ for which the total projection error is minimum.

Find the first basis vector



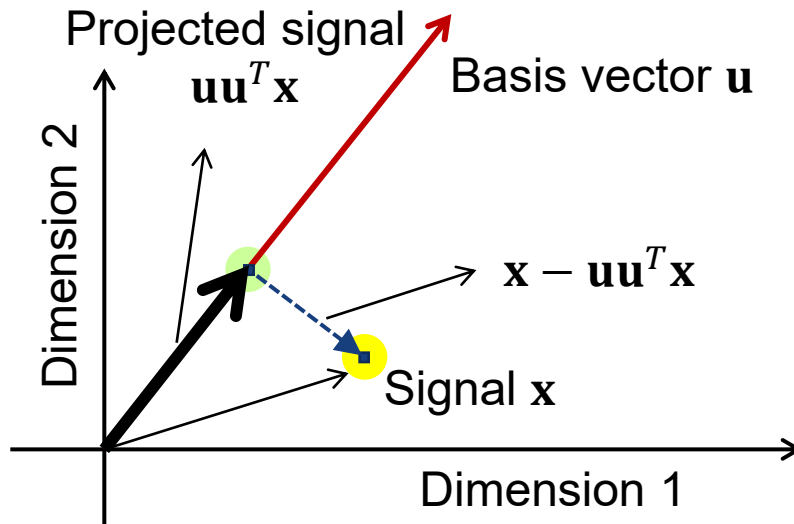
- Every signal data will suffer an error when approximated by its projection on u .
- The total length of all error lines is the *total projection error*.
- Objective: Find the best u for which the total projection error is minimum.

Find the first basis vector



- Every signal data will suffer an error when approximated by its projection on $\mathbf{u}$.
- The total length of all error lines is the *total projection error*.
- Objective: Find the best $\mathbf{u}$ for which the total projection error is minimum.

Find the first basis vector



Project a signal vector $\mathbf{x}$ on to a basis vector $\mathbf{u}$ with a unit length ($\|\mathbf{u}\| = 1$), $\hat{\mathbf{x}} = \mathbf{u}\mathbf{u}^T \mathbf{x}$

$$\begin{aligned} \text{Error} &= \|\mathbf{x} - \hat{\mathbf{x}}\|^2 = \|\mathbf{x} - \mathbf{u}\mathbf{u}^T \mathbf{x}\|^2 \\ &= (\mathbf{x} - \mathbf{u}\mathbf{u}^T \mathbf{x})^T (\mathbf{x} - \mathbf{u}\mathbf{u}^T \mathbf{x}) \\ &= \mathbf{x}^T \mathbf{x} - \mathbf{x}^T \mathbf{u}\mathbf{u}^T \mathbf{x} - \mathbf{x}^T \mathbf{u}\mathbf{u}^T \mathbf{x} + \mathbf{x}^T \mathbf{u}\mathbf{u}^T \mathbf{u}\mathbf{u}^T \mathbf{x} \\ &= \mathbf{x}^T \mathbf{x} - \mathbf{x}^T \mathbf{u}\mathbf{u}^T \mathbf{x} \end{aligned}$$

$$\boxed{\mathbf{u}^T \mathbf{u} = 1}$$

Problem statement

- Error for one signal vector $\mathbf{x}$ $\text{Error} = \mathbf{x}^T \mathbf{x} - \mathbf{x}^T \mathbf{u}\mathbf{u}^T \mathbf{x}$
- Error for all signal vectors $\mathbf{x}_i$

$$\text{Error} = \sum_i (\mathbf{x}_i^T \mathbf{x}_i - \mathbf{x}_i^T \mathbf{u}\mathbf{u}^T \mathbf{x}_i) = \sum_i \mathbf{x}_i^T \mathbf{x}_i - \sum_i \mathbf{x}_i^T \mathbf{u}\mathbf{u}^T \mathbf{x}_i$$

- Goal: Find $\mathbf{u}$ to minimize this error.



Appendix: Find the first basis vector

Find $\mathbf{u}$ to minimize the error $\sum_i \mathbf{x}_i^T \mathbf{x}_i - \sum_i \mathbf{x}_i^T \mathbf{u} \mathbf{u}^T \mathbf{x}_i$ subject to $\mathbf{u}^T \mathbf{u} = 1$

Apply Lagrange multiplier α and set derivative (with respect to $\mathbf{u}$) to be zero

unit vector

$$C = \sum_i \mathbf{x}_i^T \mathbf{x}_i - \sum_i \mathbf{x}_i^T \mathbf{u} \mathbf{u}^T \mathbf{x}_i + \alpha(\mathbf{u}^T \mathbf{u} - 1)$$

$$-2 \sum_i \mathbf{x}_i \mathbf{x}_i^T \mathbf{u} + 2\alpha \mathbf{u} = \mathbf{0} \Rightarrow \left(\sum_i \mathbf{x}_i \mathbf{x}_i^T \right) \mathbf{u} = \alpha \mathbf{u}$$

- $\mathbf{u}$: eigenvector of matrix $\sum_i \mathbf{x}_i \mathbf{x}_i^T$
- α is eigenvalue

Minimize $C = \sum_i \mathbf{x}_i^T \mathbf{x}_i - \sum_i \mathbf{x}_i^T \mathbf{u} \mathbf{u}^T \mathbf{x}_i$

$$= \sum_i \mathbf{x}_i^T \mathbf{x}_i - \sum_i \mathbf{u}^T \mathbf{x}_i \mathbf{x}_i^T \mathbf{u}$$

$$= \sum_i \mathbf{x}_i^T \mathbf{x}_i - \mathbf{u}^T \left(\sum_i \mathbf{x}_i \mathbf{x}_i^T \right) \mathbf{u} = \sum_i \mathbf{x}_i^T \mathbf{x}_i - \mathbf{u}^T \alpha \mathbf{u}$$

$$= \sum_i \mathbf{x}_i^T \mathbf{x}_i - \alpha \mathbf{u}^T \mathbf{u}$$

$$= \sum_i \mathbf{x}_i^T \mathbf{x}_i - \alpha$$

Recall $\mathbf{x}_i^T \mathbf{u} = \mathbf{u}^T \mathbf{x}_i$

Recall $\left(\sum_i \mathbf{x}_i \mathbf{x}_i^T \right) \mathbf{u} = \alpha \mathbf{u}$

Recall $\mathbf{u}^T \mathbf{u} = 1$

Choose the largest α to minimize this error

So far, we already have the first basis. To get more basis, we need to

- Get the “error” signal $\mathbf{x} - \mathbf{u}_1 \mathbf{u}_1^T \mathbf{x}$ by subtracting the first-level approximation from the original signal.
- Treat the “error signal” as a new signal and repeat the estimation on the “error” signal.
- We can continue the process until we find N basis vectors.

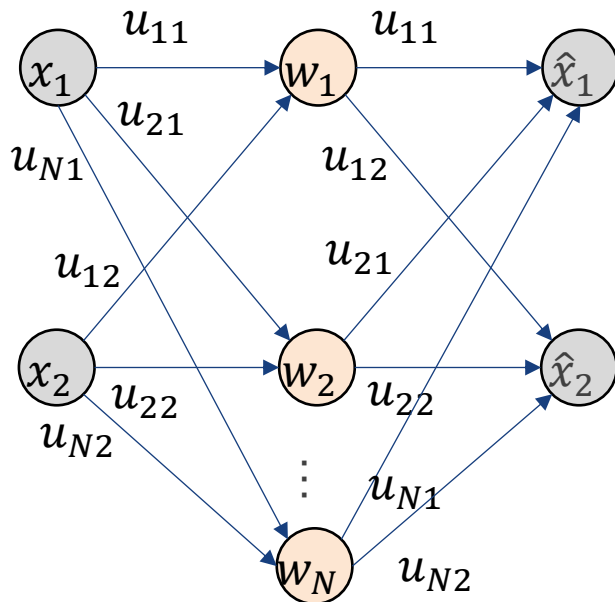
While techniques such as Principal Component Analysis (PCA) allow us to learn a complete set of basis vectors efficiently, we wish to learn an **over-complete** set of basis vectors to represent input vectors $\mathbf{x} \in \mathbb{R}^n$ (i.e. such that $k > n$). The advantage of having an over-complete basis is that our basis vectors are better able to capture structures and patterns inherent in the input data. However, with an over-complete basis, the coefficients a_i are no longer uniquely determined by the input vector $\mathbf{x}$.

Quoted from Andrew Ng’s tutorial, <http://ufldl.stanford.edu/tutorial/unsupervised/SparseCoding>

Recall: Given a multiple-dimensional signal $\mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}$,

and N basis vectors $\mathbf{u}_1 = \begin{pmatrix} u_{11} \\ u_{12} \end{pmatrix}, \mathbf{u}_2 = \begin{pmatrix} u_{21} \\ u_{22} \end{pmatrix}, \dots, \mathbf{u}_N = \begin{pmatrix} u_{N1} \\ u_{N2} \end{pmatrix}$.

- **Decompose** signal as $w_i = \langle \mathbf{x}, \mathbf{u}_i \rangle$, for example, $w_1 = \langle \mathbf{x}, \mathbf{u}_1 \rangle = x_1 u_{11} + x_2 u_{12}$, $w_2 = \langle \mathbf{x}, \mathbf{u}_2 \rangle = x_1 u_{21} + x_2 u_{22}$
- **Reconstruct (approximate)** signal as $\hat{\mathbf{x}} = \sum_{i=1}^N w_i \times \mathbf{u}_i$, for example, $\hat{x}_1 = w_1 u_{11} + w_2 u_{21} + \dots + w_N u_{N1}$, $\hat{x}_2 = w_1 u_{12} + w_2 u_{22} + \dots + w_N u_{N2}$



Signal representation learning can be interpreted as a neural network

Input

Output

Model weights

Hidden layer output

Signal

Reconstructed signal

Basis vectors

Signal representation coefficients

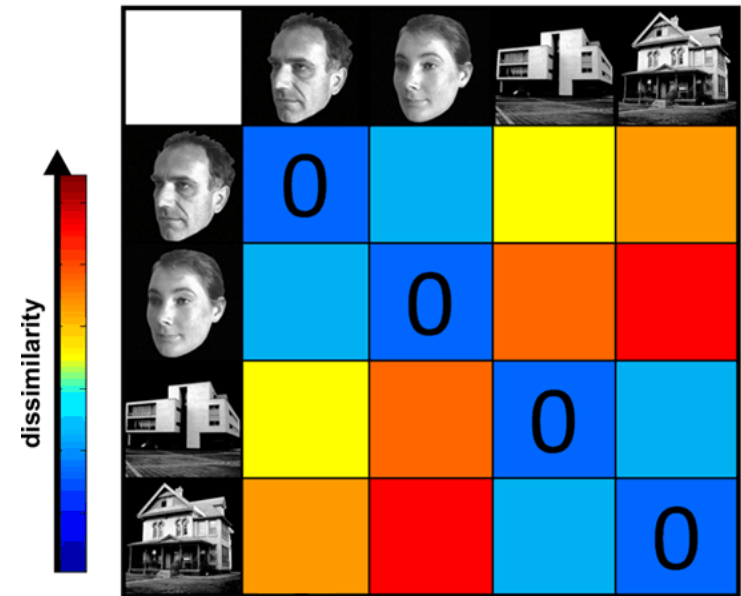
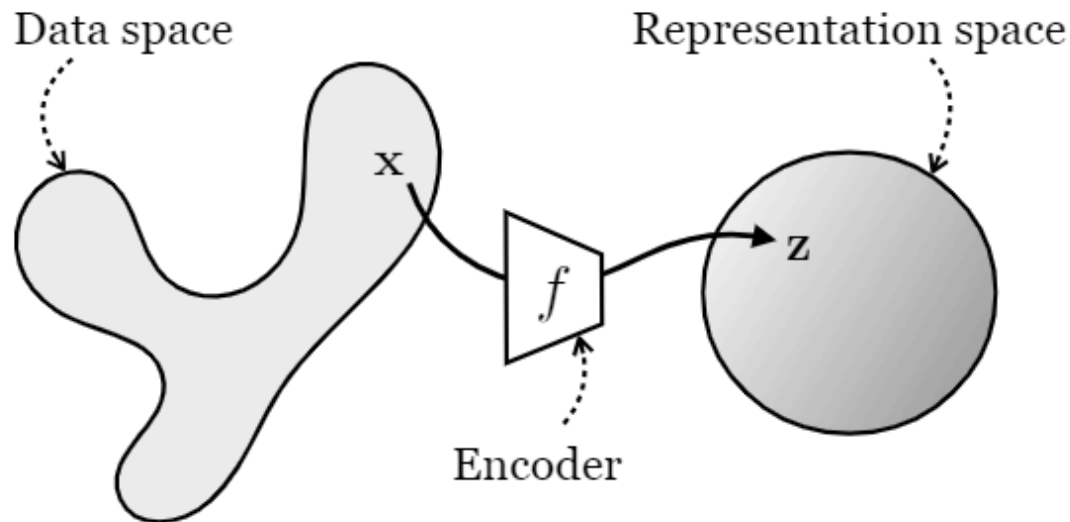
Note: Only a *linear* transformation is used in this slide.



REPRESENTATION LEARNING FOR SENSORY DATA

Representation learning: Objective

Representation learning: Learn to map from data space to abstract representations, which are typically more useful than the raw data (for analytics tasks).



Reference: Representational similarity analysis, <https://www.frontiersin.org/articles/10.3389/neuro.06.004.2008/full>

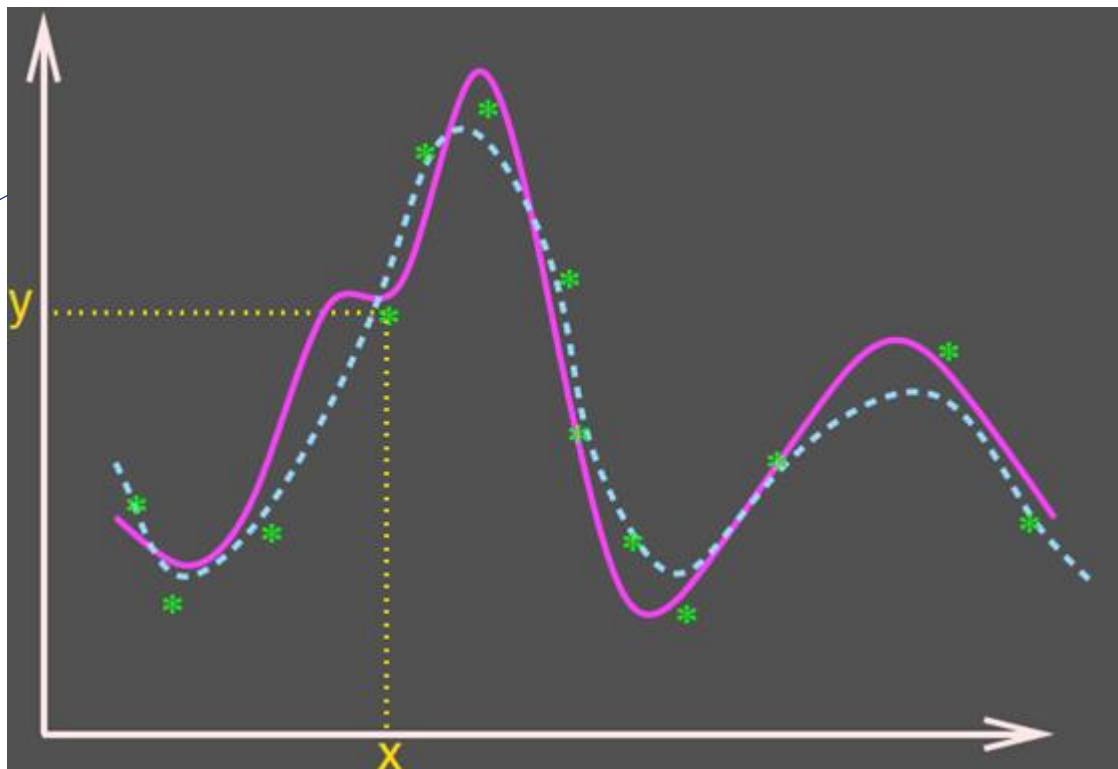


Representation learning: Considerations

1. Smoothness

- By “smoothness,” we mean that close inputs are mapped to close outputs (representations).
- To fit the data (green stars), the blue model is smoother than the purple model.

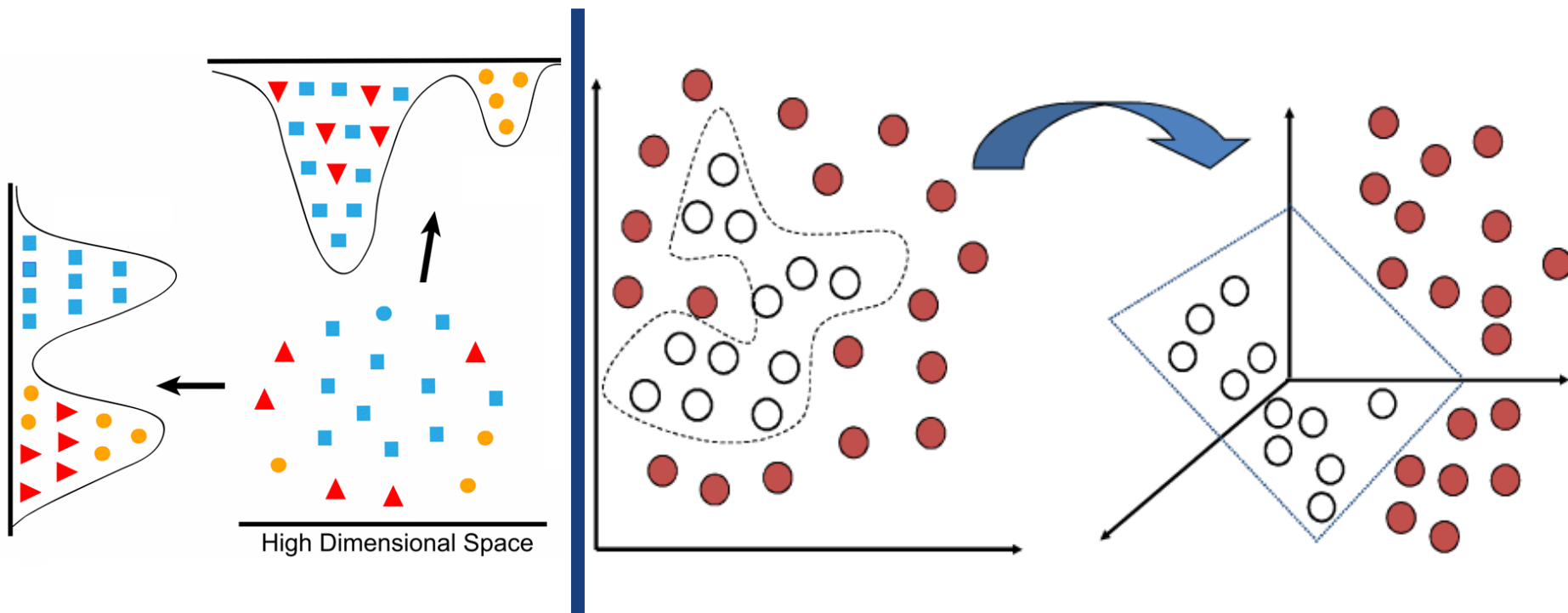
An example of model fitting
 $y = f(x)$





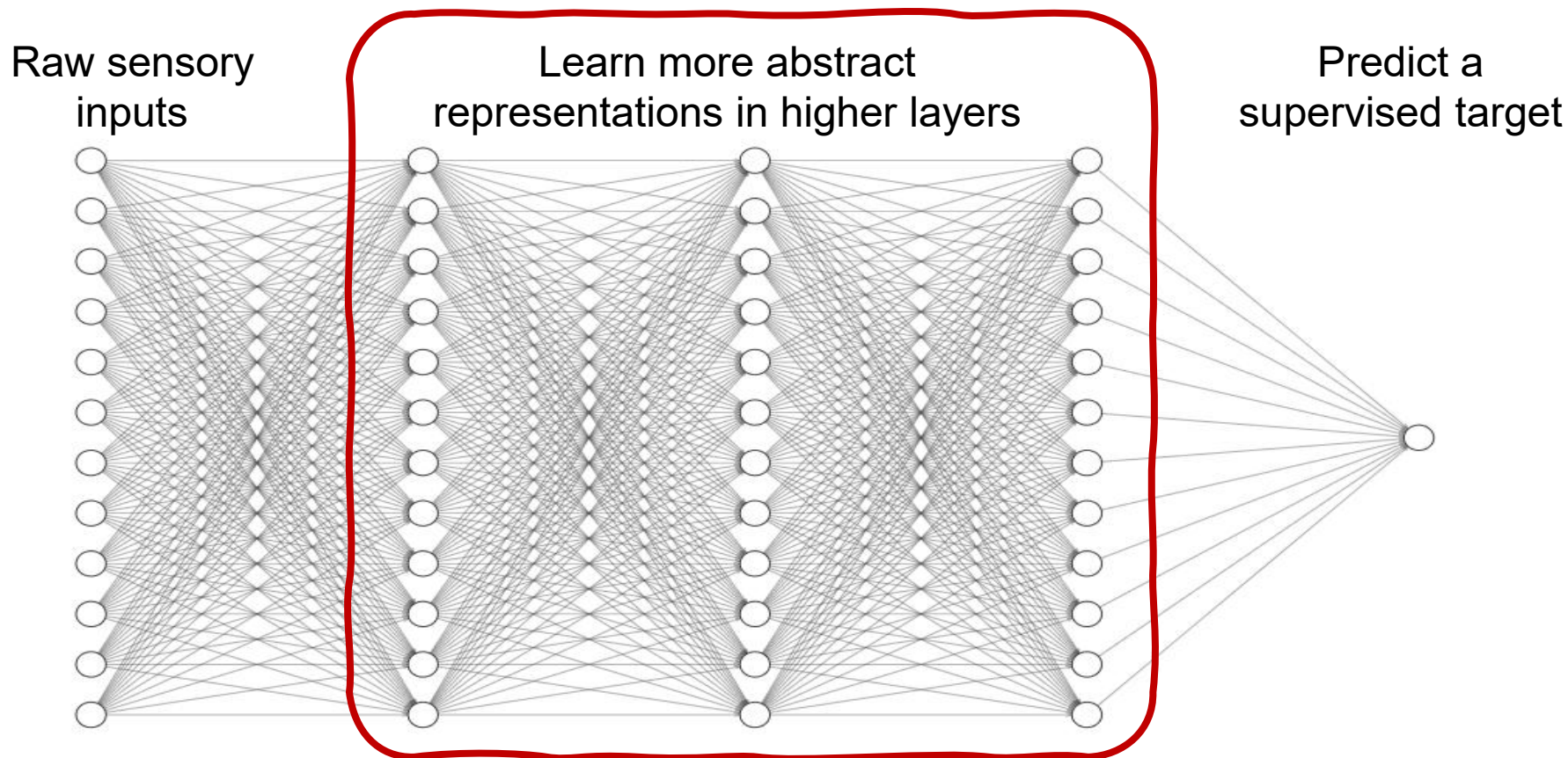
2. Dimensionality

- Sometimes, we need a representation with a **lower** dimension than the input dimension to avoid complicated calculations. But note that we should not lose too much important data.
- Sometimes, we also need to **increase** the dimensions of data to obtain richer representations.



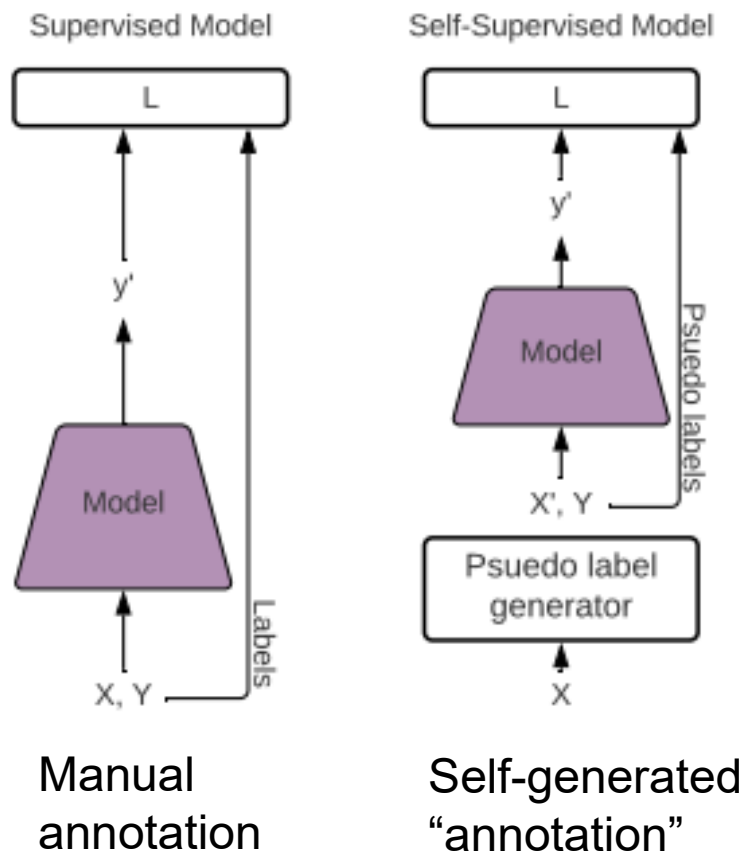
3. Depth and abstraction

A good representation captures high-level, abstract features. To achieve such representations, we can use deep architectures that reuse low-level features to produce more abstract features at higher layers.





Representation learning: Overview



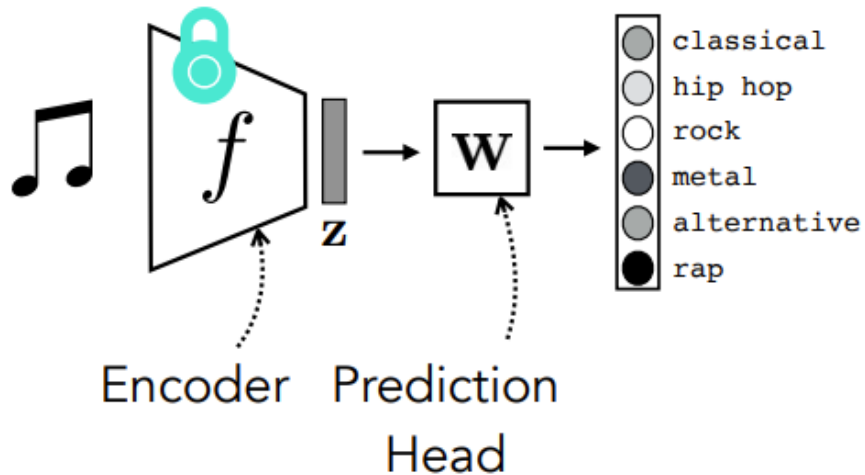
- **Supervised learning:** Supervised learning indicates learning methods using data with fine-grained human-annotated labels to train networks.
- **Self-supervised learning:** Self-supervised learning refers to learning methods where models are explicitly trained with supervisory signals that are generated from the data itself (self-supervision) by leveraging its structure.

Representation learning (supervised)

Linear adaptation: Freeze f , train a new linear mapping to new target data.

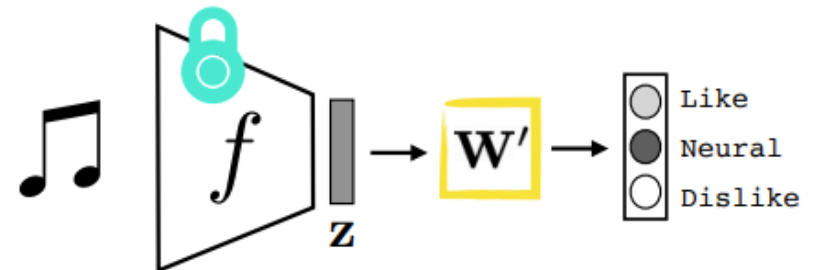
Training

Genre recognition



Adapting

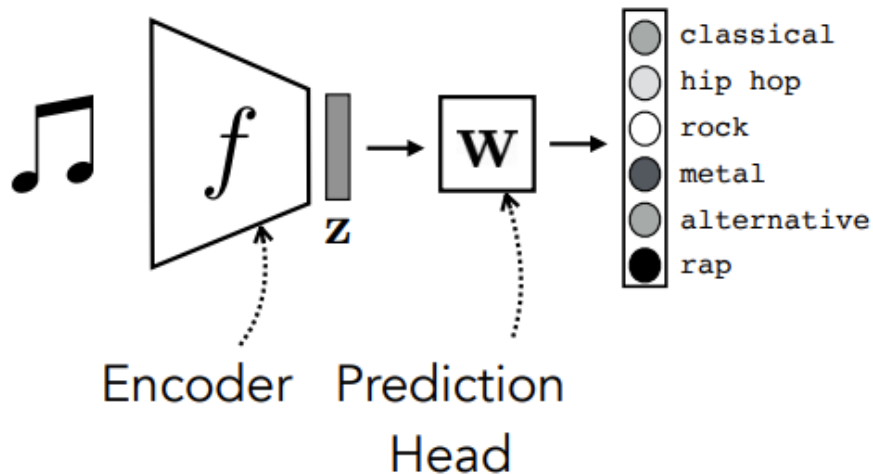
Preference prediction



Finetuning: Initialize f' as f , then continue training on new target data.

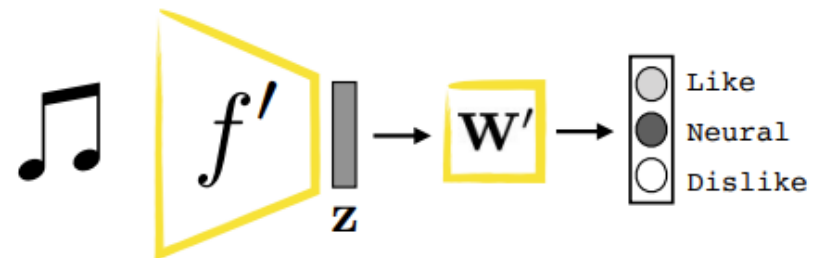
Training

Genre recognition



Adapting

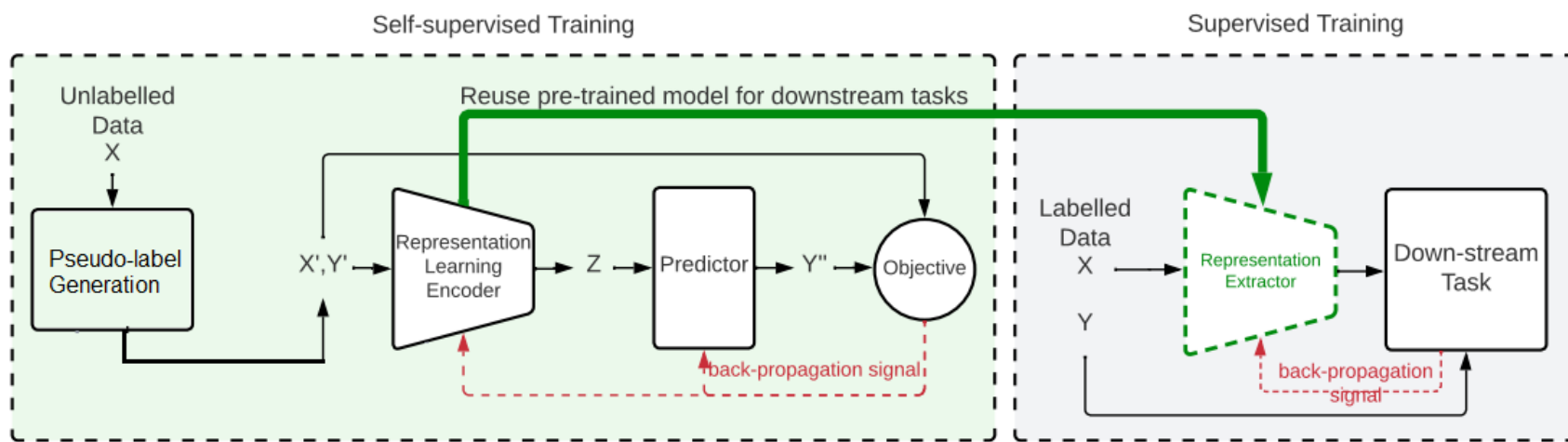
Preference prediction





Representation learning (self-supervised)

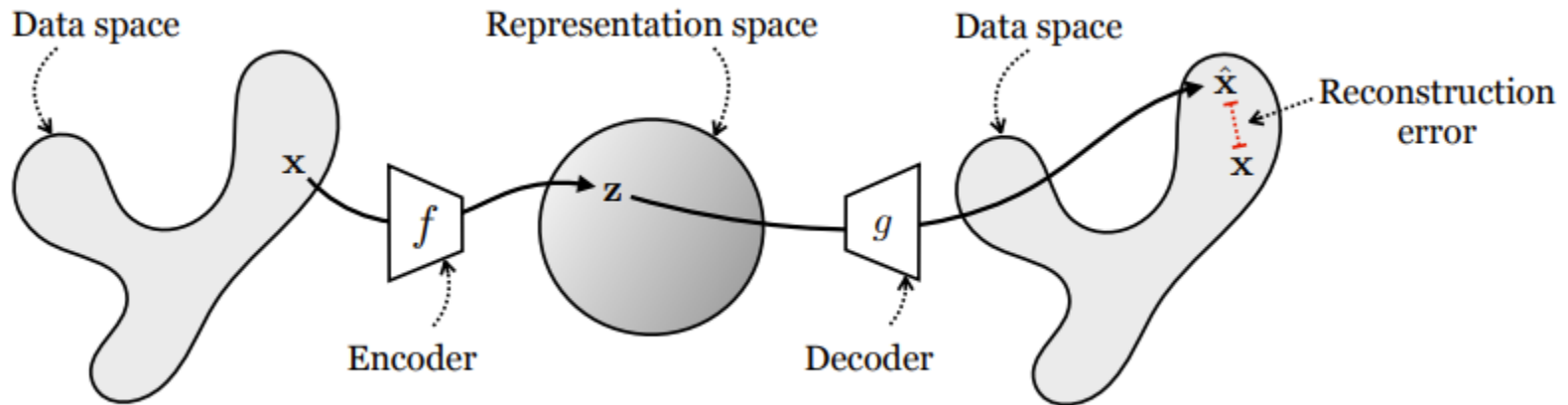
- **Pretext task:** Pretext tasks are pre-designed tasks for networks to solve, and features are learned by learning the objective functions of pretext tasks.
- **Target task (downstream task):** Target tasks can be used to evaluate the quality of features learned by self-supervised learning. These applications can benefit from the pre-trained models when training data is scarce. In general, human-annotated labels are needed to solve the target tasks.



Reference: Beyond Just Vision: A Review on Self-Supervised Representation Learning on Multimodal and Temporal Data, <https://arxiv.org/abs/2206.02353>



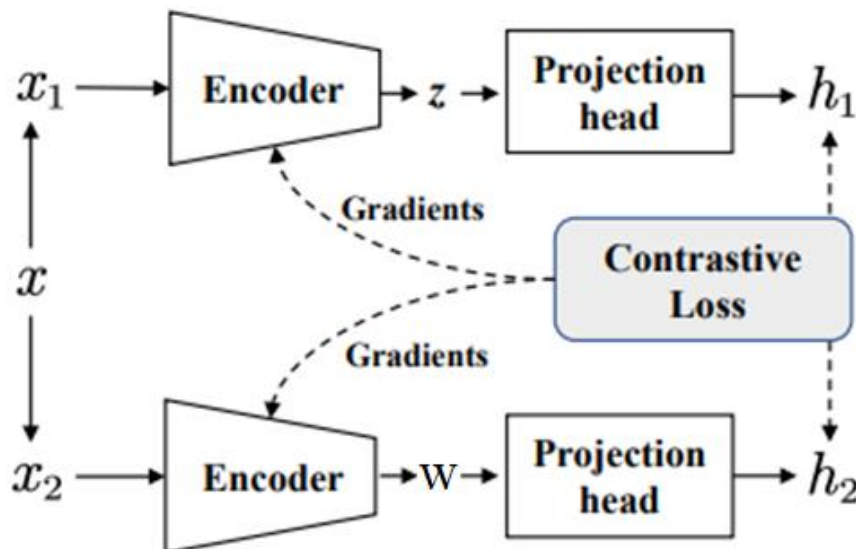
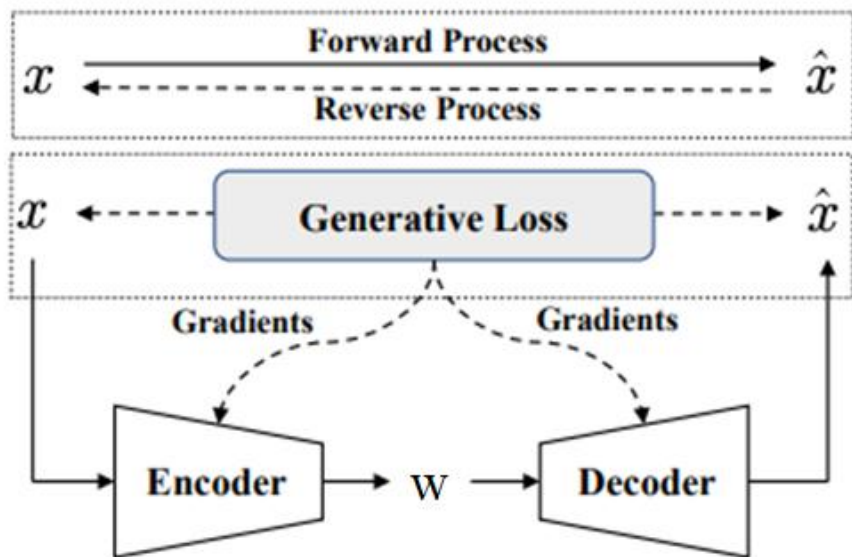
Autoencoder: Overview



- **Idea:** Train a model that is optimized (in terms of the optimal basis vectors) to output a signal as close to the input signal itself.
- This model is called an **autoencoder**, containing an encoder (signal decomposition) and a decoder (signal reconstruction).



Representation learning (self-supervised)

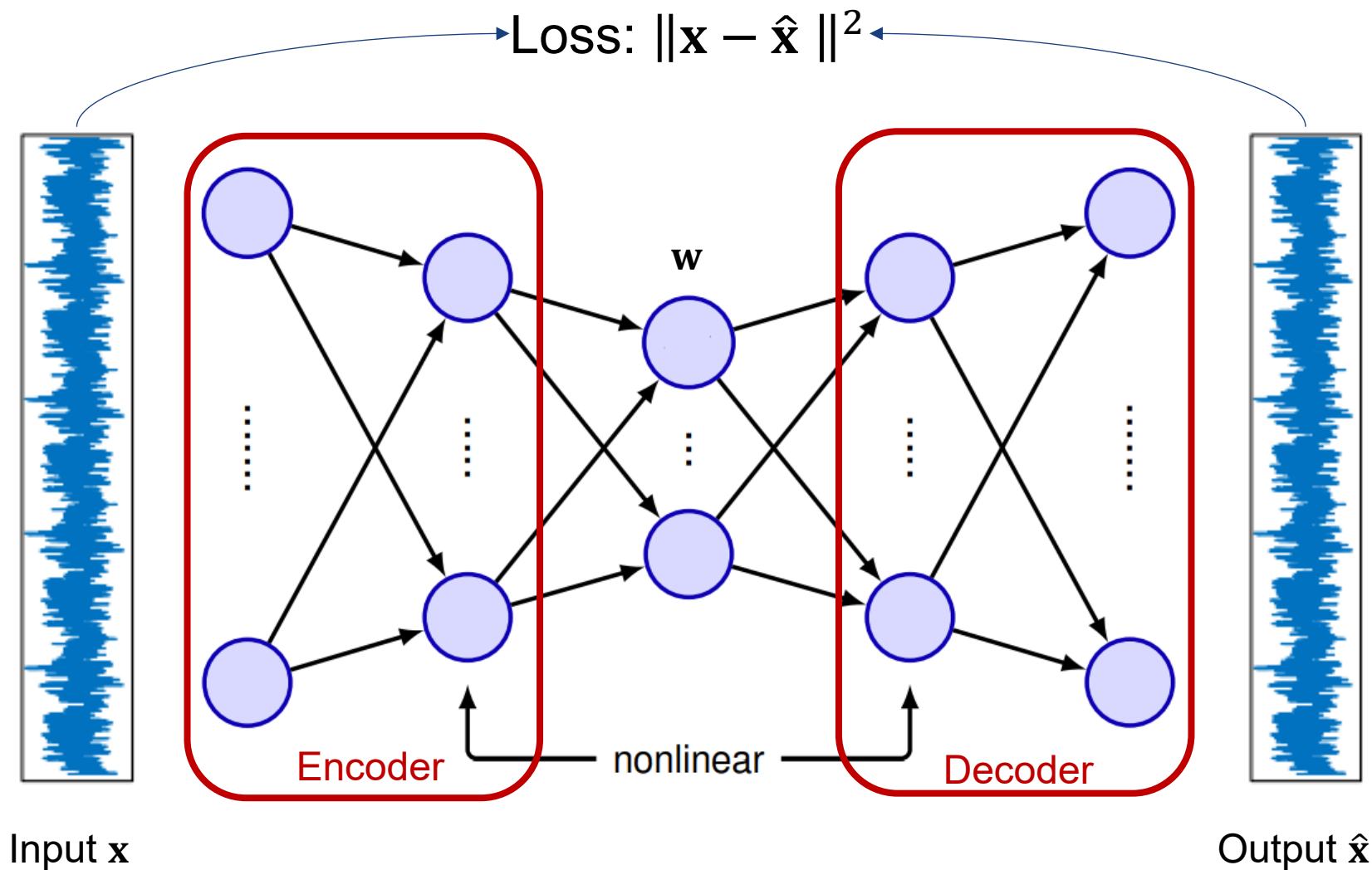


- The **generative-based** approach first uses an encoder to map the input x to the representation w , and then a decoder to reconstruct $\hat{x}$ from w . The training objective is to minimize the reconstruction error between the input x and the reconstructed input $\hat{x}$.

- The **contrastive-based** approach constructs positive and negative samples (x_1 and x_2) through data augmentation or context sampling. The model is then trained by maximizing the similarity between positive samples and the dissimilarity between negative samples.



Generative autoencoder: Fully-connected

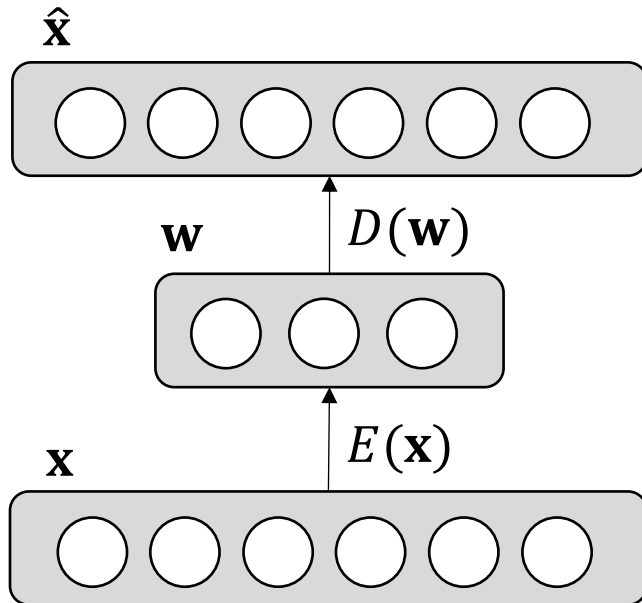




Generative autoencoder: Fully-connected

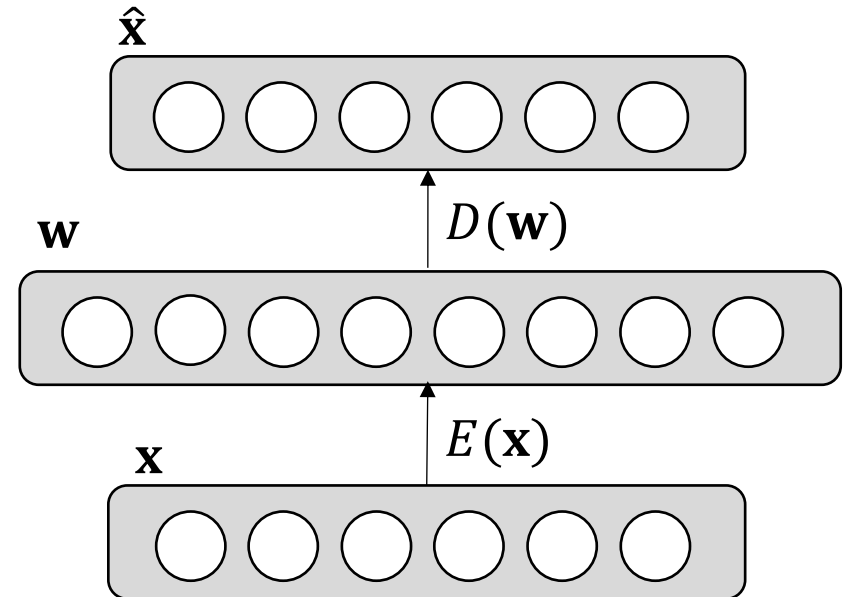
Under-complete

- The hidden layer has fewer dimensions than the input layer.
- Compresses the input.
- Hidden nodes will be good features for the training distribution.



Over-complete

- The hidden layer has more dimensions than the input layer.
- No compression in the hidden layer.
- A higher-dimensional code helps model a more complex distribution.





Generative autoencoder: Convolutional

The **MNIST** handwritten digits database.
Each image has a resolution of 28×28 .



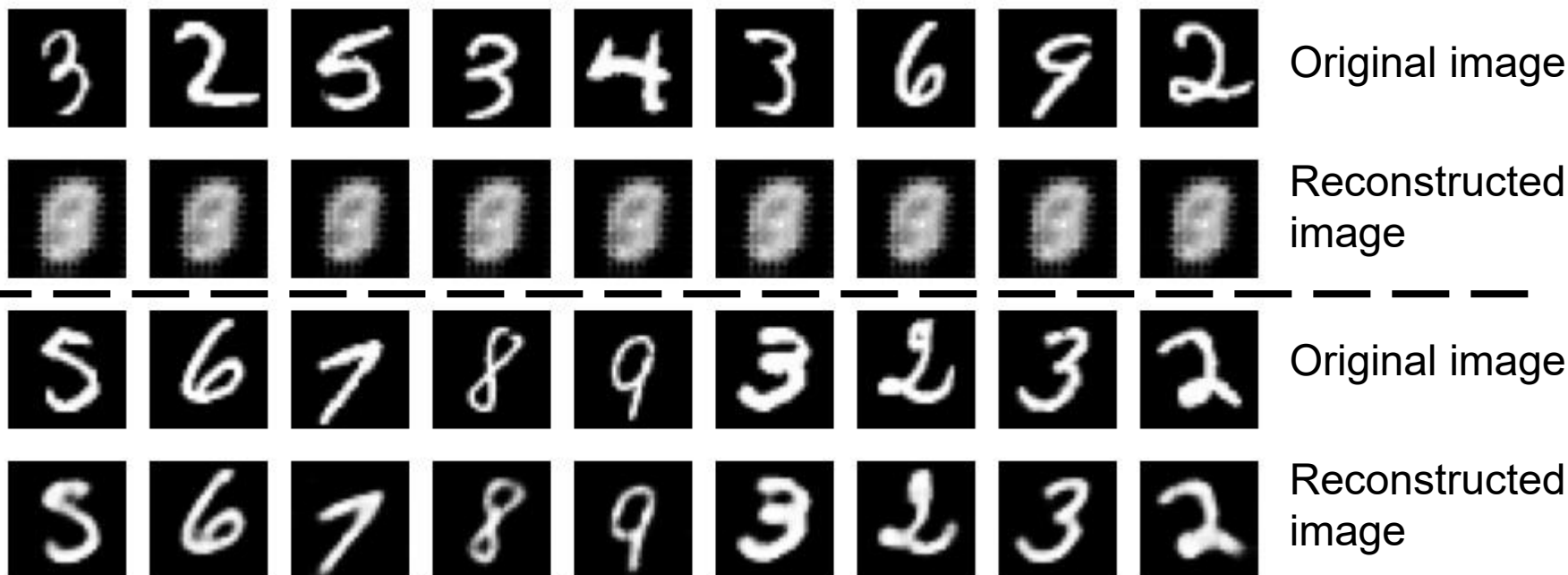
Layer (type)	Output Shape
Conv2d-1	<code>[-1, 16, 14, 14]</code>
ReLU-2	<code>[-1, 16, 14, 14]</code>
Conv2d-3	<code>[-1, 32, 7, 7]</code>
ReLU-4	<code>[-1, 32, 7, 7]</code>
Conv2d-5	<code>[-1, 64, 1, 1]</code>
ConvTranspose2d-6	<code>[-1, 32, 7, 7]</code>
ReLU-7	<code>[-1, 32, 7, 7]</code>
ConvTranspose2d-8	<code>[-1, 16, 14, 14]</code>
ReLU-9	<code>[-1, 16, 14, 14]</code>
ConvTranspose2d-10	<code>[-1, 1, 28, 28]</code>
Sigmoid-11	<code>[-1, 1, 28, 28]</code>

```
nn.MSELoss()  
batch_size = 64  
Epoch = 20  
torch.optim.Adam()  
lr = 1e-3  
weight_decay = 1e-5
```



Generative autoencoder: Convolutional

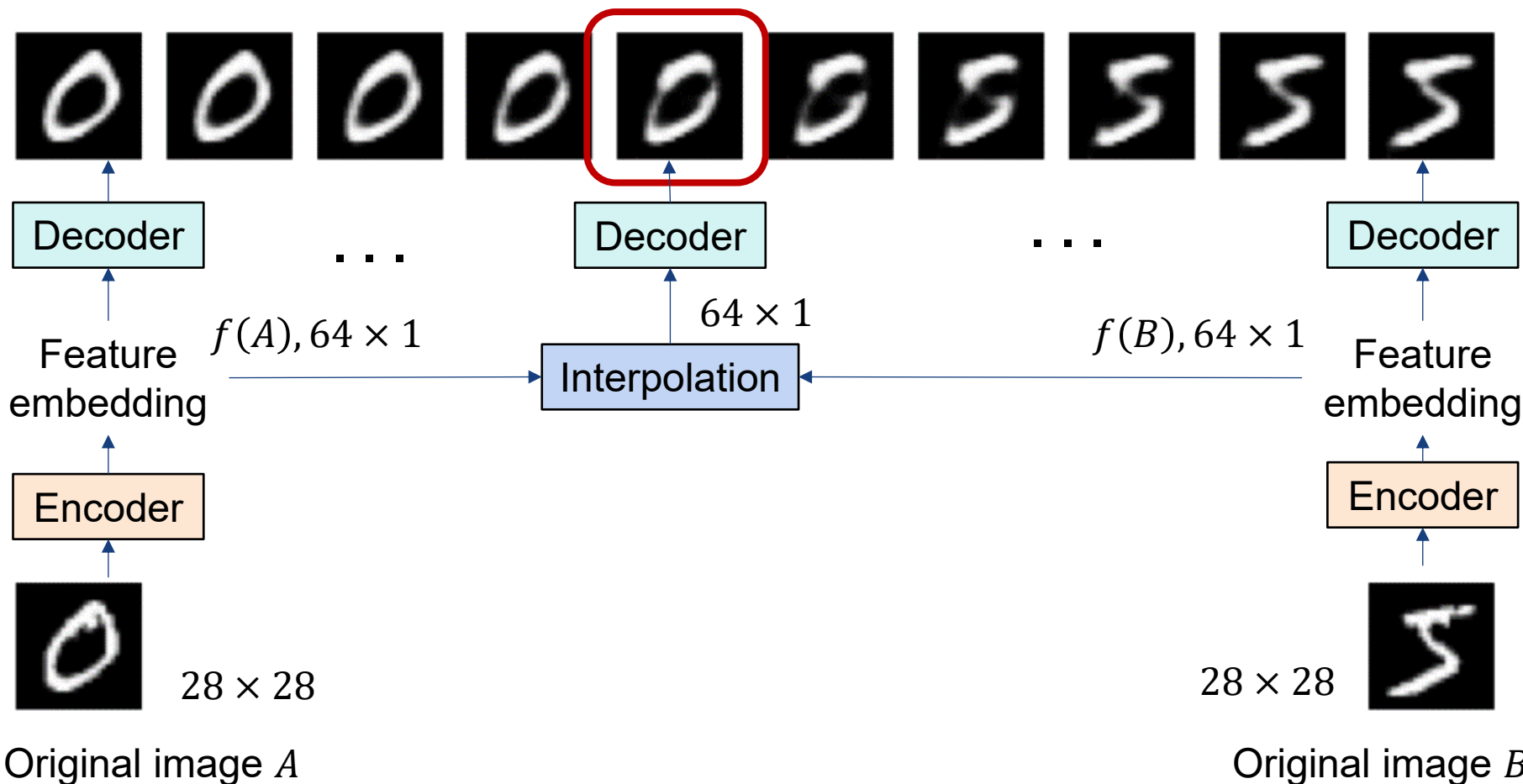
Experiment: Original images and reconstruction images with **different numbers of training iterations**, including small (top row) and large (bottom row).





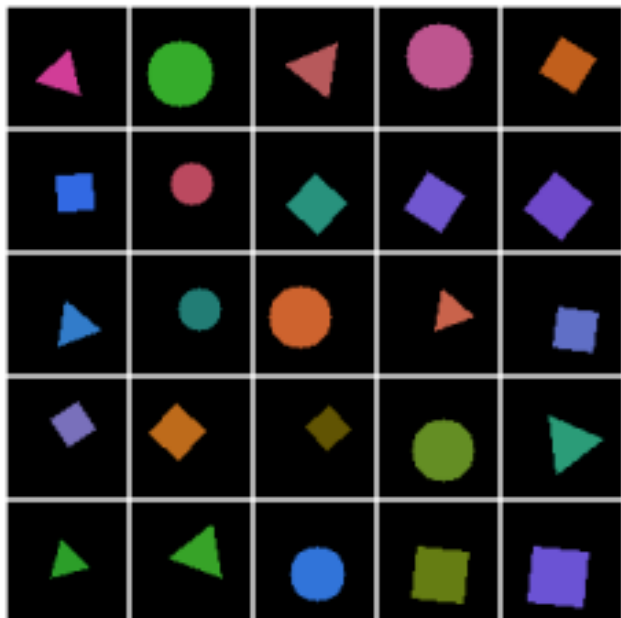
Generative autoencoder: Convolutional

Reconstructed image $\hat{A}$ Reconstructed image using **interpolated** embedding (between two feature embedding vectors $f(A)$, $f(B)$) Reconstructed image $\hat{B}$





Generative autoencoder: Convolutional



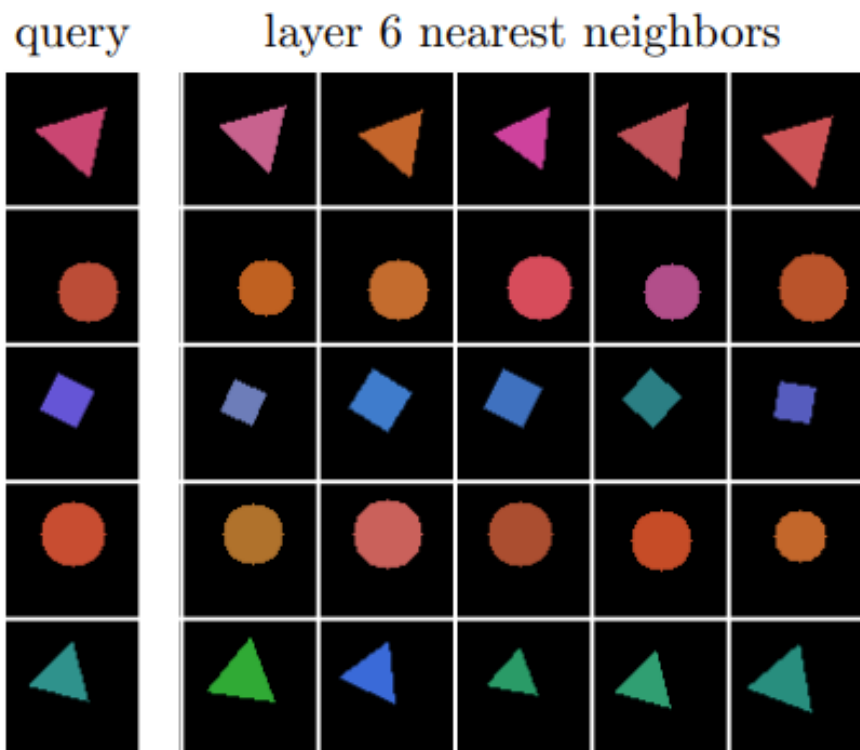
Setup: For the autoencoder architecture, we use a convolutional encoder and decoder, each with 6 *conv* layers interspersed with *relu* nonlinearities and a 128-dimensional bottleneck. We train this autoencoder for 20,000 steps of stochastic gradient descent, using the *Adam* optimizer with a batch size of 128.

8 color classes and 3 shape classes

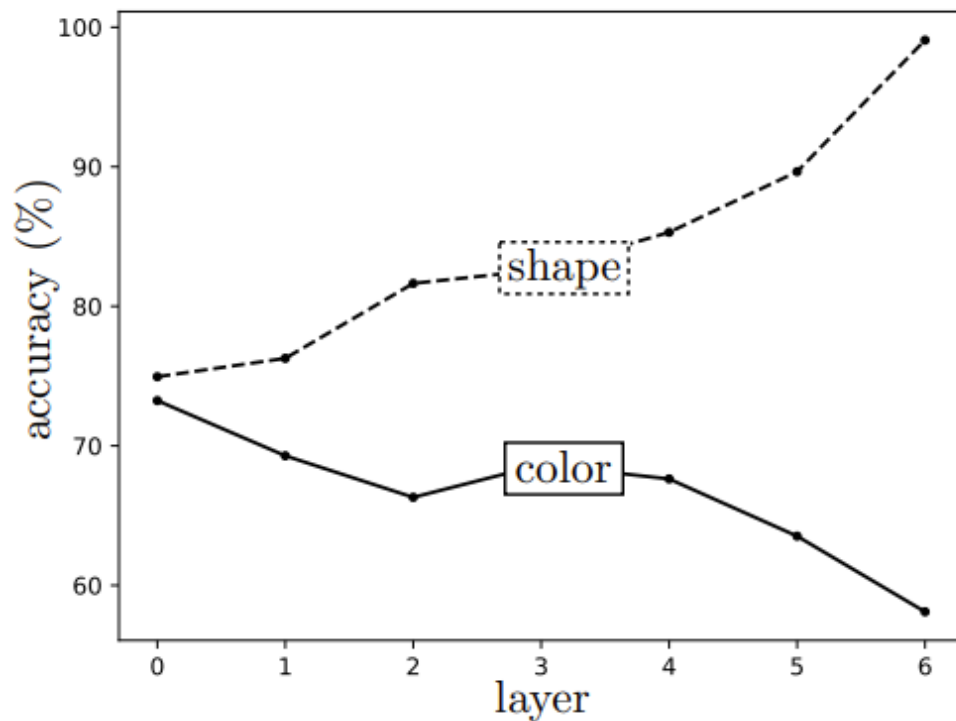


Generative autoencoder: Convolutional

Experiment 1: For a given query, visualize the images in the dataset whose embeddings are nearest neighbors to the query's embedding.



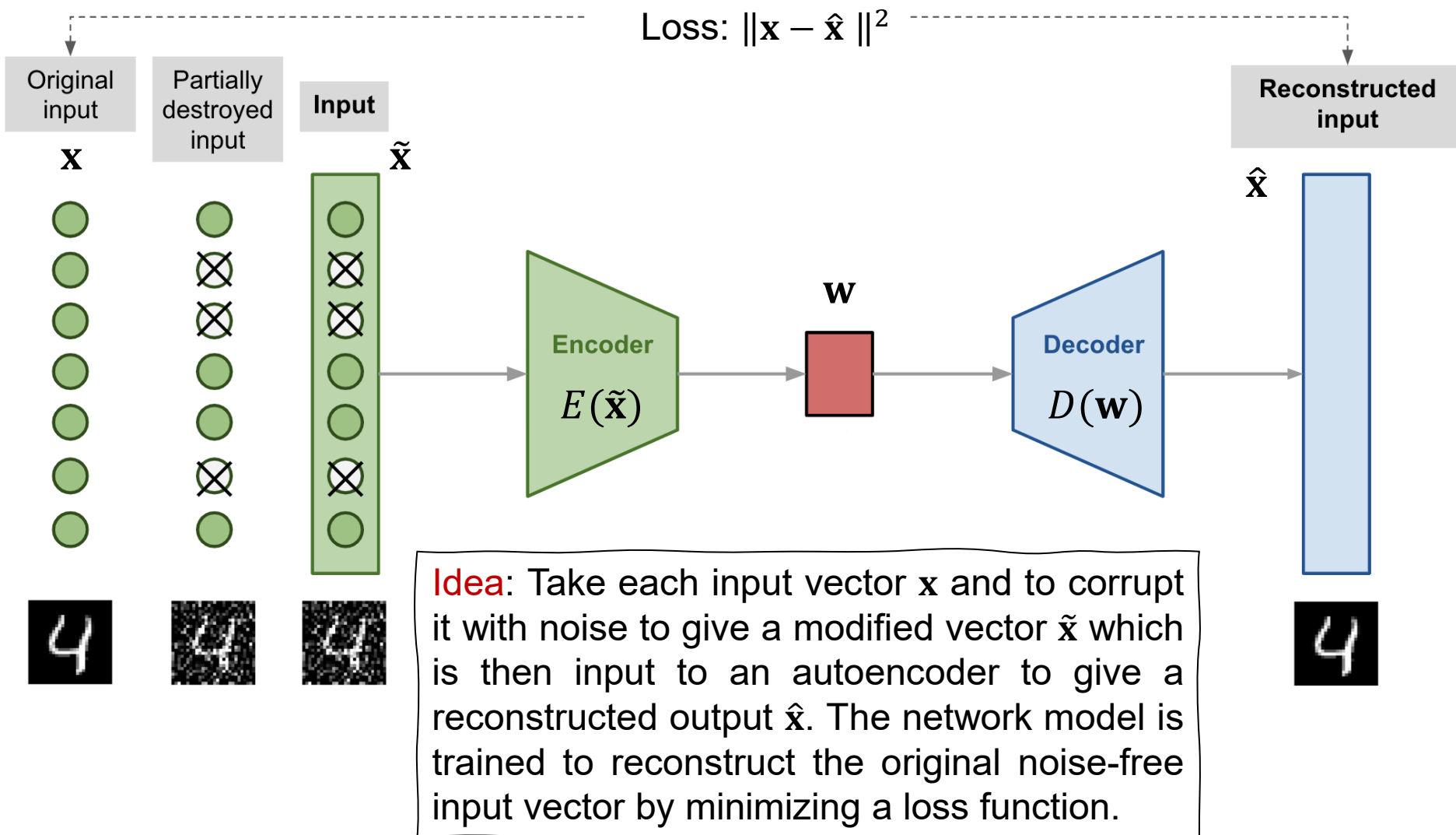
Experiment 2: Measure the accuracy of a one-nearest-neighbor classifier using the features of each layer. Layer 0 means the color values of the raw images.



Reference: Representation learning, <https://phillipi.github.io/6.s898/>

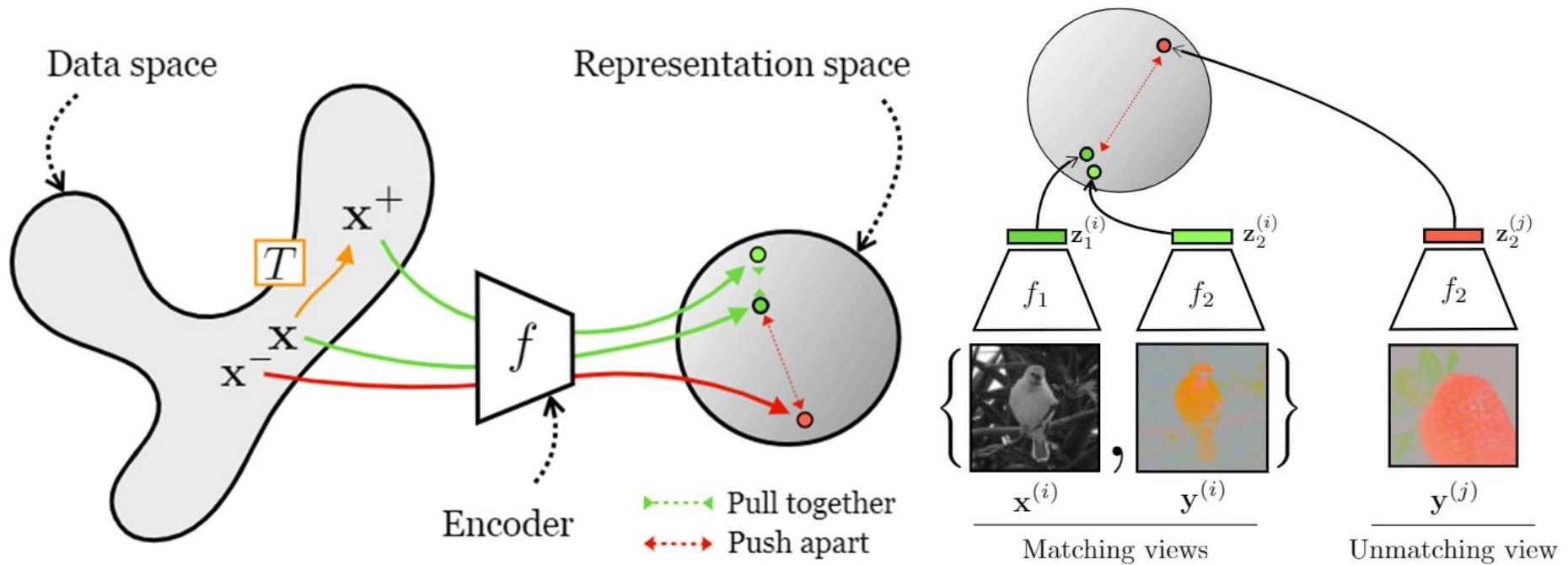


Generative autoencoder: Denoising

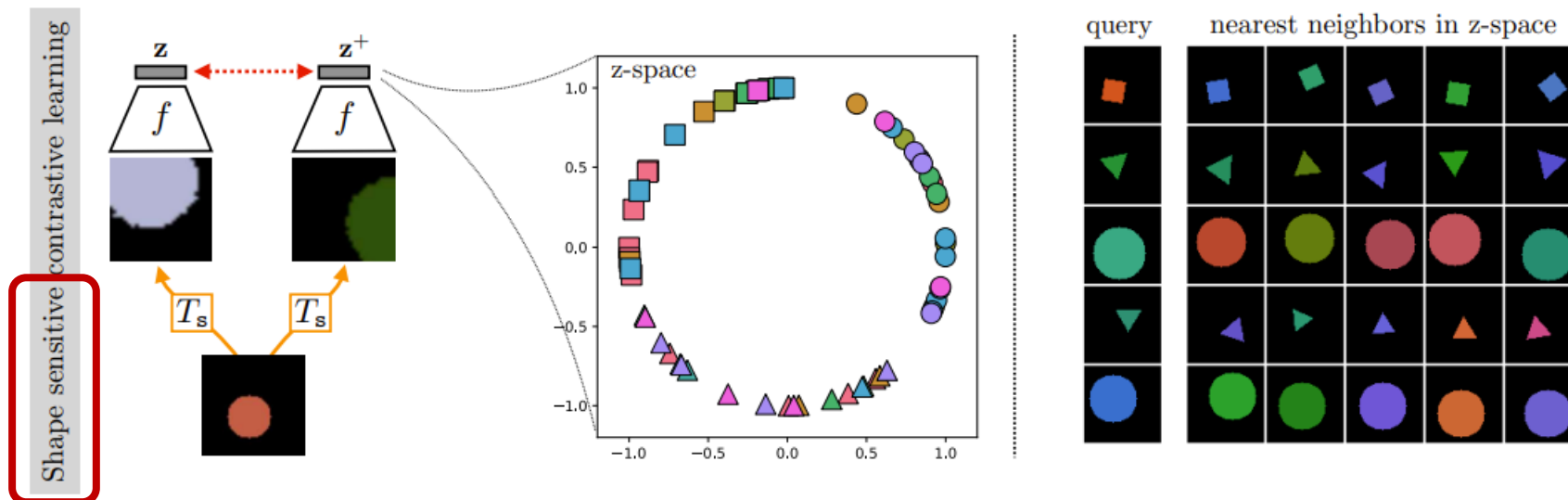
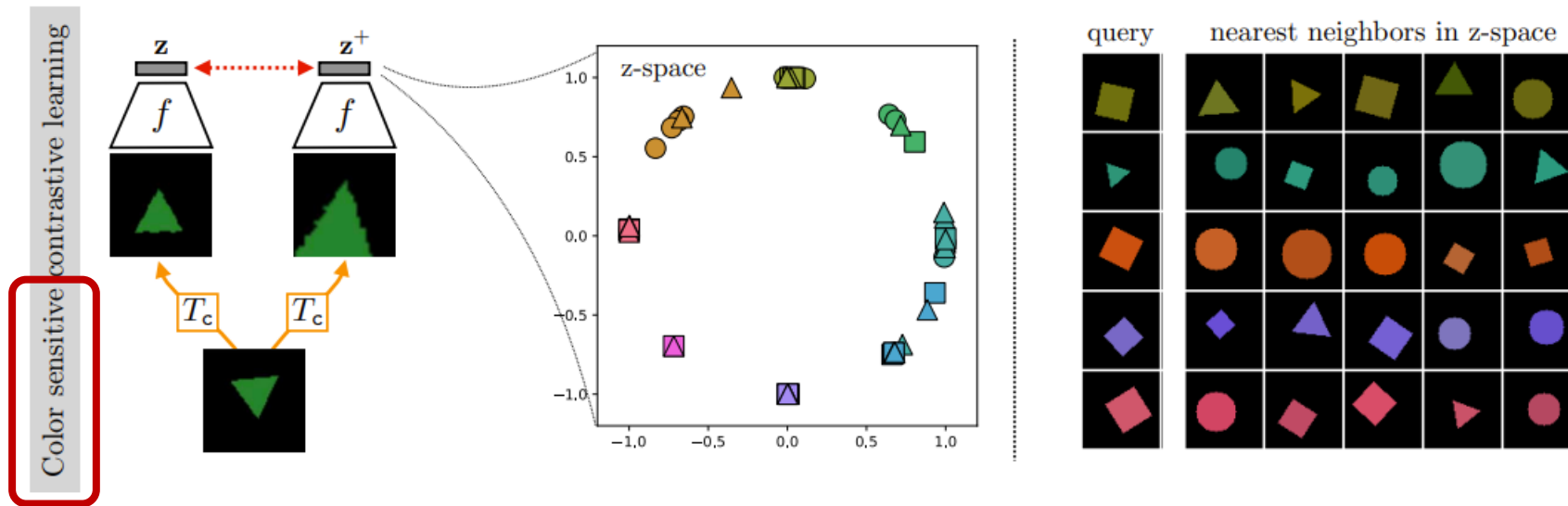


Contrastive autoencoder

Objective: To supervise what information should be embedded (or thrown away).



Contrastive autoencoder



Reference: Representation learning, <https://phillipi.github.io/6.s898/>

TECHNOMETRICS

VOL. 11, No. 1

FEBRUARY 1969

Procedures for Detecting Outlying Observations in Samples

FRANK E. GRUBBS*

U. S. Army Aberdeen Research and Development Center
Aberdeen Proving Ground, Maryland 21005

An outlier is an observation that *deviates so much from other* observations as to arouse suspicions that it was generated by a different mechanism.

Quoted from D. M. Hawkins, *Identification of Outliers* (1980)

Binary output

- Each test instance is given a *normal* or *anomaly* label

Score output

- Each test instance is assigned an anomaly score
- Allows outputs to be ranked
- Requires an additional threshold

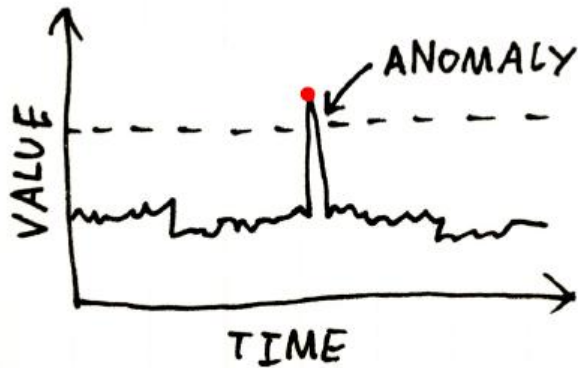
Applications

- Medical Diagnosis
- Fraud Detection
- Manufacturing Defect Detection

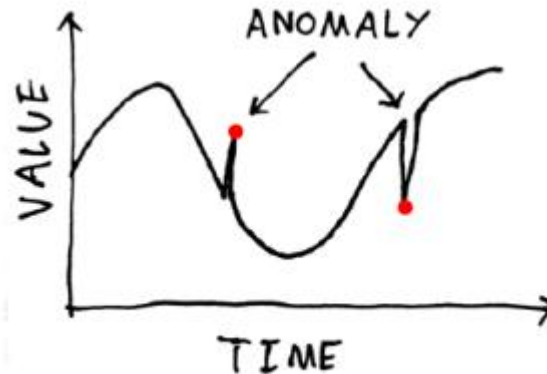
Reference: F. E. Grubbs, "Procedures for detecting outlying observations in samples," Technometrics, Vol. 11, No. 1, 1969, pp. 1-21, <https://www.jstor.org/stable/1266761>



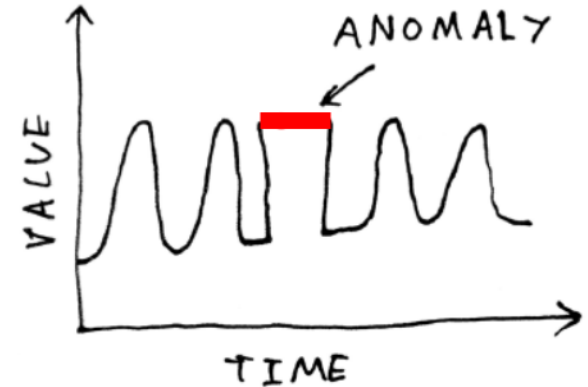
Anomaly detection



Point anomaly



Contextual anomaly



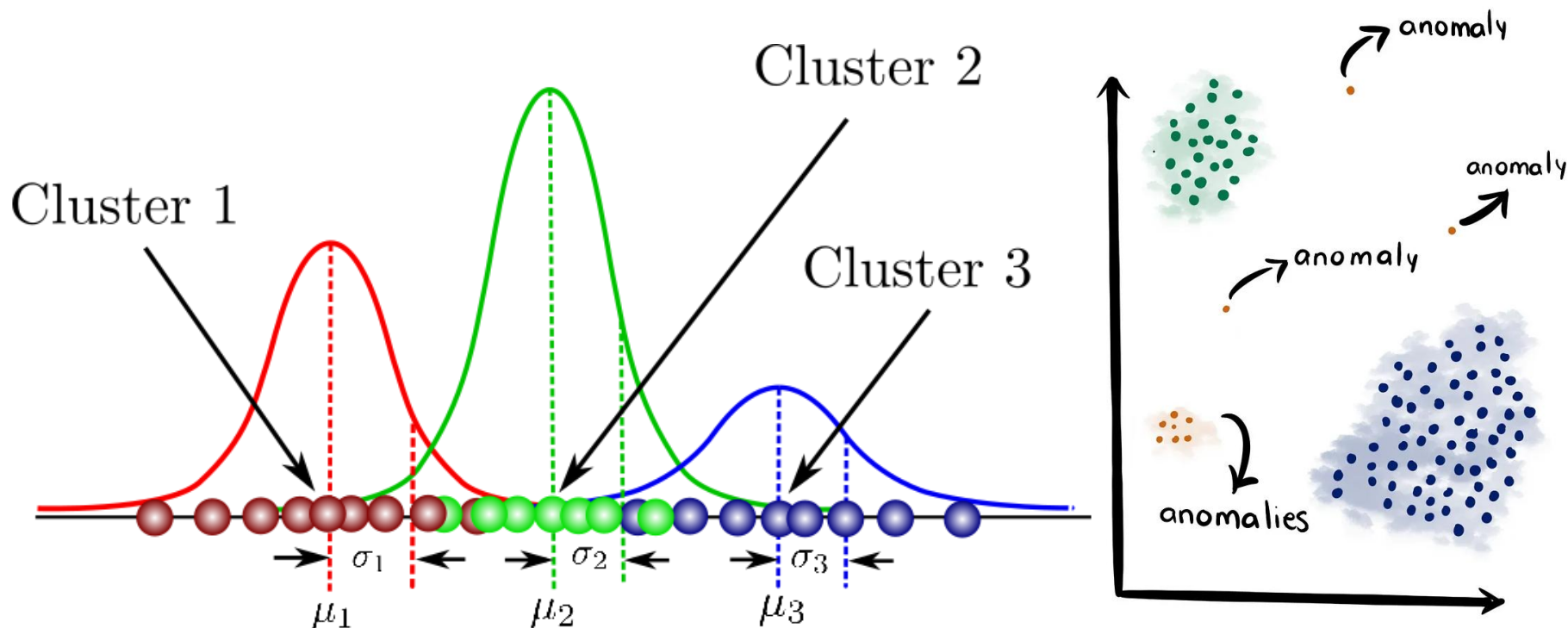
Collective anomaly

- **Point anomaly:** A data point is considered a global outlier if its value is far outside the entirety of the data set in which it is found.
- **Contextual anomaly:** A data point is considered a contextual outlier if its value significantly deviates from the rest of the data points in the same context.
- **Collective anomaly:** A subset of data points within a data set is considered anomalous if those values, as a collection, deviate significantly from the entire data set, but the values of the individual data points are not themselves anomalous in either a contextual or global sense.



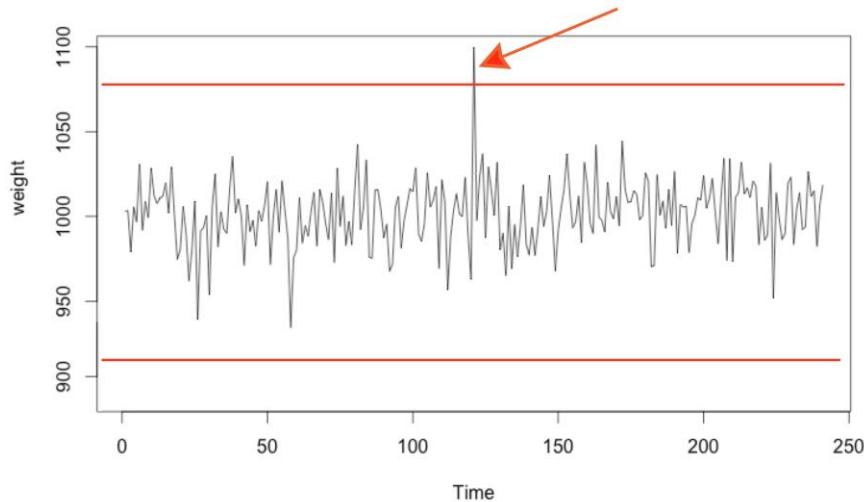
Anomaly detection: Geometrical

Geometrical	Assumption	Anomaly scoring
	Normal data points belong to a cluster (or lie close to its centroid), while anomalies do not belong to any clusters (or sparse clusters).	Distance from the nearest cluster centroid.

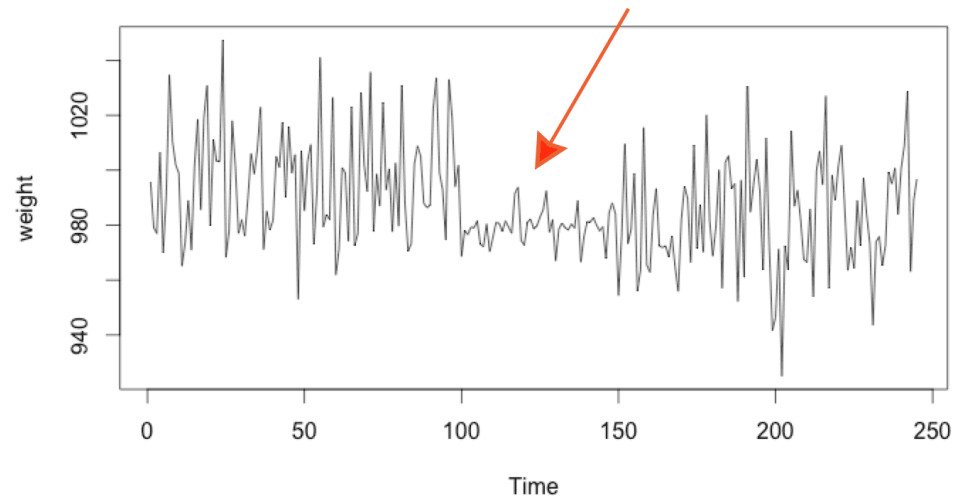


Anomaly detection: Statistical

Statistical	Assumption	Anomaly scoring
	Given an assumed stochastic model, normal data instances fall in high-probability regions of the model, while abnormal data lie in low-probability regions.	Probability that a data point lies in a high-probability region in the assumed distribution.



Detect **outlier values**, when the value deviates too much from the mean $\pm 4\sigma$.



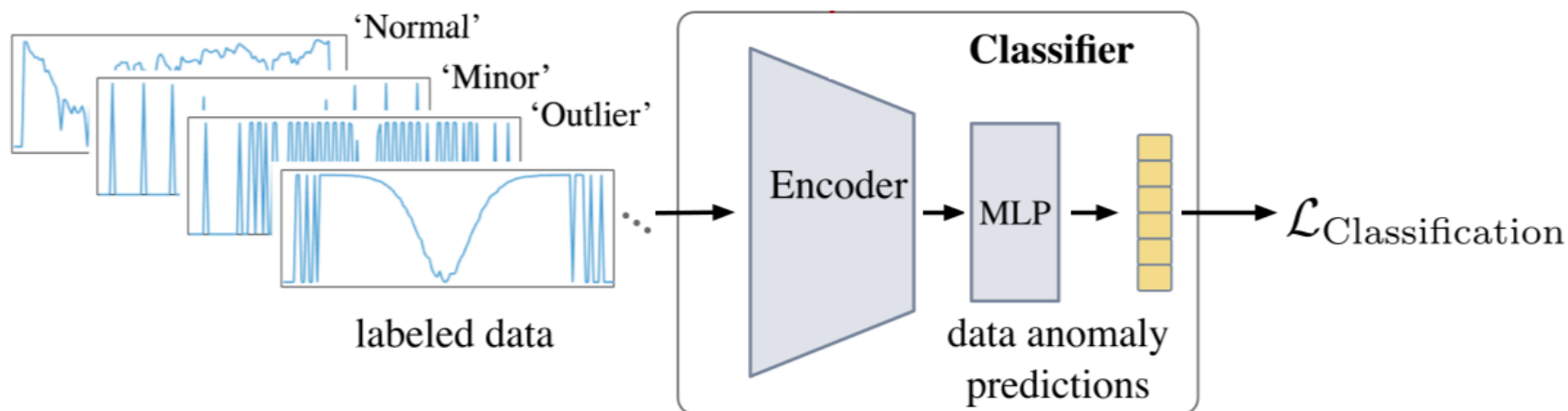
Detecting a **change** in the Normal distribution using a sliding window.



Anomaly detection: Classification

(supervised) Classification	Assumption	Anomaly scoring
	A classifier can be learned to distinguish between normal and abnormal. Labelled data exists for the normal and abnormal classes.	A measure of the classifier score (likelihood) that the test data belongs to the normal class.

Machine learning-based classifier



Anomaly detection: Autoencoder

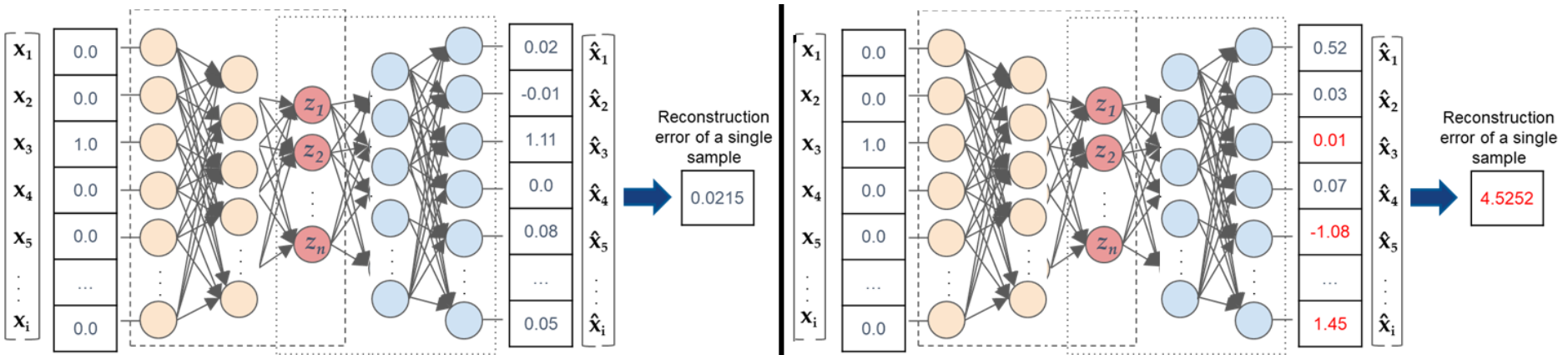
(unsupervised)
Deviation

Assumption

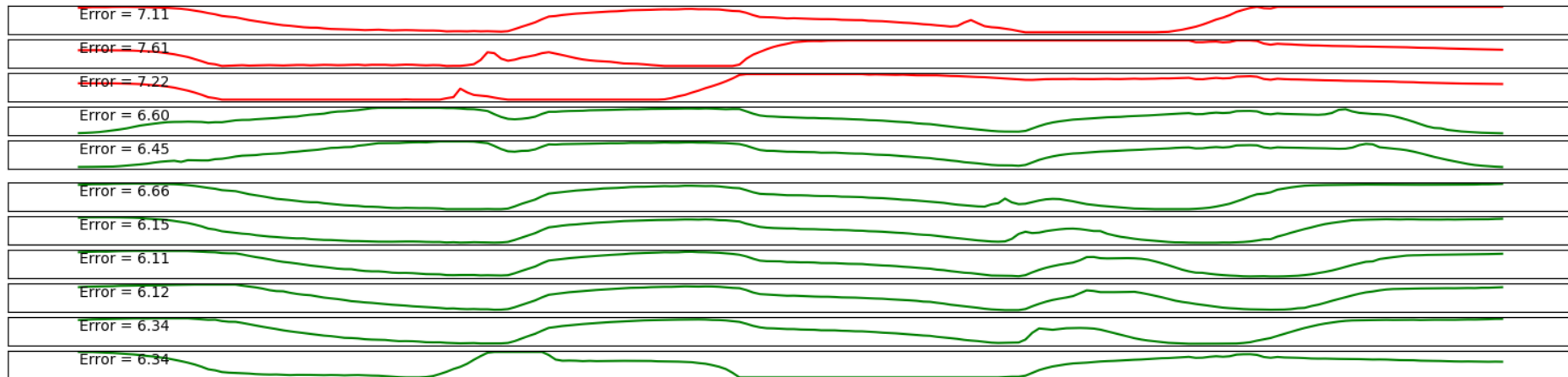
Anomalies are very rare compared to normal data, so they will have larger reconstruction errors.

Anomaly scoring

Reconstruction error obtained from the autoencoder model.



- Train an auto-encoder on the training data.
- Evaluate it on the validation data and plot the reconstruction error.
- Choose a threshold that determines whether an error value is an outlier (anomaly) or not.



Thank you!

Dr TIAN Jing

Email: tianjing@nus.edu.sg